Repositories & API Keys — Info Document

Page Overview

Purpose

The Repositories & API Keys page is the single surface for inspecting every scholarly repository `resmon` can query and for managing the API keys those repositories require. It lists the full catalog served by the backend, exposes per-repository metadata (subject coverage, endpoint, rate limit, upstream policy, credential requirement), and lets the user store, replace, or remove an API key for each key-gated repository in either the local OS keyring or — when signed in — the user's cloud account.

Primary User Flows

- Browse the repository catalog with one row per active repository.
- Click a repository name (or caret) to expand a details panel for that row, and use Expand All / Collapse All to reveal or hide every detail panel at once.
- Type an API key into the inline input for a key-gated repository and press Enter to save it.
- Click Clear on a row with a saved key to delete that key from the active scope.
- Toggle the scope selector between **This device (keyring)** and **Cloud account** to choose which credential store read/write operations target.
- Sign in to the cloud account (via the global header) to enable the cloud scope; signing out while viewing the cloud scope snaps the selector back to local.

Inputs and Outputs

- **Inputs to the page:** the repository catalog (`GET /api/repositories/catalog`), local credential presence (`GET /api/credentials`), and — when the cloud scope is active — cloud credential presence (`GET /api/v2/credentials` , via the authenticated cloud client).
- **User inputs:** the scope choice, row expansion state, and per-row API-key text entered into each `ApiKeyField` .
- **Outputs (side effects):** writes to the OS-native keyring (local scope) via `PUT/DELETE /api/credentials/{name}` , or writes to the encrypted cloud credential store (cloud scope) via `PUT/DELETE /api/v2/credentials/{name}` .

- **Outputs (displayed):** a catalog table with a 12-character mask (`************`) whenever a key is saved, an Expand All / Collapse All control, and per-row inline error messages.

Known Constraints

- **Endpoints never return key values.** Both `GET /api/credentials` and `GET /api/v2/credentials` return presence-only maps; the raw key is never sent back to the UI. Saved keys always display as the fixed 12-character mask.
- **Cloud scope requires sign-in.** If the user is not signed in, selecting the cloud tab shows a muted "Sign in to manage credentials stored in your cloud account" message and the page does not call the cloud API.
- **Credential-name whitelist.** `PUT /api/credentials/{key_name}` rejects any name that is not in the union of `catalog_credential_names()` , `AI_CREDENTIAL_NAMES` , and `SMTP_CREDENTIAL_NAMES` with HTTP 400.
- **Key-less repositories.** arXiv, CrossRef, OpenAlex, bioRxiv, medRxiv, DOAJ, EuropePMC, DBLP, HAL, and PubMed do not require a key; their API-key column shows "Not required".
- **Key-required repositories.** CORE, IEEE Xplore, and NASA ADS are skipped at sweep time when no key is stored. Deep Dive accepts an ephemeral, per-execution key via the `push_ephemeral` / `pop_ephemeral` mechanism instead of requiring persistence.
- **No test/validate button on this page.** The `POST /api/credentials/validate` endpoint exists but is invoked from the Settings → AI panel, not from the Repositories page; saving a key here does not perform a live probe.

Frontend

Route and Main Component

- Route: `/repositories` (registered in `App.tsx`).
- Main component: `RepositoriesPage` (`resmon_scripts/frontend/src/pages/RepositoriesPage.tsx`).

Child Components and Hooks

- `PageHelp` — collapsible help card with sections "What this page does", "Scope selector", and "Key-less repositories".
- `RepoCatalogTable` (`components/Repositories/RepoCatalogTable.tsx`) — renders Expand All / Collapse All controls and the three-column table (Repo / Subject Coverage / API Key); manages per-row expansion, inline key values, and inline error state.

- `ApiKeyField` (`components/Repositories/ApiKeyField.tsx`) — the single-line input that displays the 12-character mask when a key is present, submits on Enter, clears on Escape, and exposes a Clear button when a stored key exists.
- `RepoDetailsPanel` (`components/Repositories/RepoDetailsPanel.tsx`) — rendered inside the expanded row; shows description, endpoint, query method, rate limit, upstream policy, parallel-safety, notes, credential name, website, and registration URL for the repository.
- `repositoriesApi` (`api/repositories.ts`) — wraps `apiClient` for the local scope and `cloudClient` for the cloud scope (`/api/v2/credentials` with JWT).
- `useAuth()` — used to read `isSignedIn` , which gates the cloud scope.
- React hooks: `useState` , `useEffect` , `useCallback` .

UI State Model

- `catalog: RepoCatalogEntry[]` — the full catalog returned by `GET /api/repositories/catalog` .
- `localPresence: CredentialPresenceMap` — `{ credential_name: { present: bool } }` for the OS keyring.
- `cloudPresence: CredentialPresenceMap` — the same shape for the cloud store; normalized in `normalizeCloudPresence` from the raw `{name: bool}` wire format.
- `scope: 'local' | 'cloud'` — active credential scope; forced back to `'local'` whenever `isSignedIn` becomes `false` .
- `loading: boolean` — true until the initial `Promise.all([getCatalog, getCredentialsPresence])` resolves.
- `error: string` — last error message to surface above the table.
- Inside `RepoCatalogTable` : `expanded: Set<string>` (row slugs), `inlineValues: Record<slug, string>` (uncommitted key text), and `errors: Record<slug, string>` (per-row inline errors).
- Inside `ApiKeyField` : `focused` , `editing` — drive the mask vs. live-value display logic.

Key Interactions and Events

- **Initial load:** `useEffect` fires `Promise.all([repositoriesApi.getCatalog(), repositoriesApi.getCredentialsPresence()])` ; on resolution `catalog` and `localPresence` are set and `loading` is cleared. A `cancelled` flag in cleanup prevents state updates after unmount.
- **Scope switch to cloud (signed in):** a second `useEffect` lazily fires `refreshCloud()` the first time the scope becomes `'cloud'` while `isSignedIn` is true.
- **Sign-out while viewing cloud:** the first `useEffect` forces `setScope('local')` so the page never shows stale cloud presence.

- **Expand / collapse:** clicking a repo name (or pressing Enter / Space on the row toggle) toggles that slug in `expanded`. Expand All sets `expanded` to every catalog slug; Collapse All clears it.
- **Save key (Enter in `ApiKeyField`):** `handleSave` trims the inline value, skips empty input, and calls `repositoriesApi.saveCredential` (local) or `repositoriesApi.putCloudCredential` (cloud) with the trimmed value. On success the inline value is cleared, the row's error is cleared, and `onPresenceRefresh()` re-fetches the active scope's presence map — which flips the row's display to the 12-character mask.
- **Clear key (Clear button):** `handleClear` calls `repositoriesApi.deleteCredential` (local) or `repositoriesApi.deleteCloudCredential` (cloud), then `onPresenceRefresh()`.
- **Escape / blur with no edits:** `ApiKeyField` resets `editing` and `value` so the mask re-appears.

Error and Empty States

- **Loading:** while the initial catalog + presence fetch is in flight, the page renders only "Loading repository catalog…" as muted text.
- **Load failure:** an error from `getCatalog`, `getCredentialsPresence`, or `getCloudCredentials` is captured into `error` and rendered as a `.form-error` banner above the table. The table still renders from whatever catalog data was obtained (empty if none).
- **Unsigned cloud view:** when `scope === 'cloud' && !isSignedIn`, the page shows "Sign in to manage credentials stored in your cloud account." and does not call the cloud API.
- **Key-less row:** the API Key cell renders "Not required" (muted) and `ApiKeyField` is not mounted.
- **Save / Clear failure:** the thrown error's `message` is written into the per-slug entry of `errors` and shown as `.form-error` text directly beneath the row's `ApiKeyField`.

Backend

API Endpoints

Handlers live in `resmon_scripts/resmon.py` unless noted otherwise.

- `GET /api/repositories/catalog` — returns the static repository catalog by calling `catalog_as_dicts()` from `implementation_scripts/repo_catalog.py`. Never returns secrets.
- `GET /api/credentials` — returns `{name: {"present": bool}}` for the sorted union of `catalog_credential_names() | AI_CREDENTIAL_NAMES | SMTP_CREDENTIAL_NAMES`. Each value is `get_credential(name) is not None`; the raw credential is never read into the response.
- `PUT /api/credentials/{key_name}` — body `{value: string}`. Rejects with HTTP 400 if `key_name` is not in the allowed union above. On success calls

`store_credential(key_name, body.value)` and returns `{success: true}`.

- `DELETE /api/credentials/{key_name}` — calls `delete_credential(key_name)` and returns `{success: true}`.
- `POST /api/credentials/validate` — body `{provider, key, base_url?}`. Calls `validate_api_key(provider, key, base_url)` and returns `{valid: bool}`. Exists but is **not** wired to the Repositories page UI; the Repositories page does not render a Test button.
- **Cloud scope (only reached when signed in, via `cloudClient`):**
  - `GET /api/v2/credentials` — presence map as `{name: bool}`, normalized in the frontend to `{name: {present: bool}}`.
  - `PUT /api/v2/credentials/{name}` — stores the cloud credential via envelope encryption.
  - `DELETE /api/v2/credentials/{name}` — removes the cloud credential.

Request/Response Patterns

- **Catalog:** `apiClient.get<RepoCatalogEntry[]>('/api/repositories/catalog')`. Each entry includes `slug`, `name`, `description`, `subject_coverage`, `endpoint`, `query_method`, `rate_limit`, `client_module`, `api_key_requirement` ∈ { `none`, `required`, `optional`, `recommended` }, `credential_name | null`, `website`, `registration_url | null`, `placeholder`, and optional `upstream_policy`, `parallel_safe`, `notes`. Rate-limit strings match `.ai:/prep/repos.csv` (e.g., arXiv `0.33 req/s (1 per 3 s)`, CrossRef `10.0 req/s (polite pool recommended)`, IEEE Xplore `0.2 req/s (1 per 5 s); per-key plan limits`).
- **Presence (local):** `GET /api/credentials` → `{name: {present: bool}}`. Never contains the key text.
- **Presence (cloud):** `GET /api/v2/credentials` → `{name: bool}`; the frontend normalizer converts to the local shape.
- **Store:** `PUT /api/credentials/{name}` with body `{value: <string>}`. The 400 error returned for an unknown `key_name` is `"Unknown credential name: <name>"`.
- **Delete:** `DELETE /api/credentials/{name}` always returns `{success: true}`; internal `delete_credential` swallows `keyring.errors.PasswordDeleteError` when the key is absent.
- **Validate (not used by this page):** `POST /api/credentials/validate` performs a minimal provider probe listed in `_VALIDATION_ENDPOINTS` — e.g., `GET https://api.openai.com/v1/models`, `GET https://api.anthropic.com/v1/models` with `x-api-key` + `anthropic-version`, `GET https://api.core.ac.uk/v3/search/works?q=test&limit=1` with `Authorization: Bearer`, `GET https://api.adsabs.harvard.edu/v1/search/query?q=test&rows=1` with `Authorization: Bearer`,

`GET https://api.springernature.com/meta/v2/json?q=test&s=1&p=1` with `api_key` query param, etc. Returns `False` on any non-200, network, timeout, or transport error and never logs the key.

Persistence Touchpoints

- **OS keyring (local scope):** all local credential writes route through `credential_manager.store_credential`, which calls `keyring.set_password(_SERVICE, key_name, value)` where `_SERVICE = APP_NAME ("resmon")`. Reads use `get_credential` / `get_password`; deletes use `delete_credential` / `delete_password`. The logger records only the credential name and service, never the value.
- **Cloud credential store:** `PUT/DELETE /api/v2/credentials/{name}` on the cloud service persist the key under envelope encryption (per ADQ-11: per-user DEK wrapped by a KMS-held KEK). The page never sees plaintext back from the server.
- **No SQLite write from this page.** The local SQLite database (`app_settings`, routines, configurations, etc.) is not mutated by any action on the Repositories & API Keys page; all writes go to the keyring or to the cloud credential store.

Execution Side Effects

- Saved keys are consumed at query time by each `BaseAPIClient` subclass via `credential_manager.get_credential_for(exec_id, key_name)`, which checks the per-execution ephemeral store first and then the persisted keyring. This is what lets Deep Dive accept a one-shot key while routines rely on persisted keys.
- `push_ephemeral(exec_id, creds)` registers per-execution credentials in an in-memory `_EPHEMERAL_CREDENTIALS[exec_id]` dict — never written to disk, never logged. `pop_ephemeral(exec_id)` removes the entry at execution teardown so the key does not outlive the run.
- Upstream 429 / transient-error handling is implemented at the HTTP-client layer (`implementation_scripts/api_base.py`), which marks `{429, 500, 502, 503, 504}` as transient codes and applies backoff retries inside `safe_request`. Catalog entries surface this in their `upstream_policy` and `notes` fields (e.g., CORE: "Registered key ~10 req/s; ~10k/day. Respect Retry-After on 429"; PLOS: "If you see 429s lower the limiter to 0.2 req/s."; Semantic Scholar: "Expect 429s without a key in bursts."). The Repositories page itself does not render live rate-limit telemetry.
- No email, report generation, cloud sync upload, scheduler mutation, or routine fire is triggered from this page. The only side effects are credential store writes and the subsequent presence re-fetch.

Repositories & API Keys — Info Document

Page Overview

Purpose

Primary User Flows

Inputs and Outputs

Known Constraints

Frontend

Route and Main Component

Child Components and Hooks

UI State Model

Key Interactions and Events

Error and Empty States

Backend

API Endpoints

Request/Response Patterns

Persistence Touchpoints

Execution Side Effects